Solving the Incompressible Navier Stokes Equation with Stabilizing terms using the FEniCSx programming software

Bibek Shrestha

MANE, Rensselaer Polytechnic Institute, 110 8th St, Troy, 12180, NY, USA.

Contributing authors: bbkshrestha25@gmail.com;

Abstract

This project aimed to develop a comprehensive approach for solving the compressible Navier-Stokes equations with stabilizing terms using the FEniCSx program. Initially, the methodology was applied to the incompressible Navier-Stokes equations. The weak formulation of the incompressible Navier-Stokes equations was derived, incorporating stabilization terms such as SUPG, PSPG, and LSIC. The formulation was discretized in time using the generalized-alpha method. This approach was implemented to solve the flow around a cylinder problem at a Reynolds number (Re) of 100 over a fixed time interval. The results were tallied with that of the DFG benchmark 2D-3 to check for the validity.

1 Introduction

1.1 Background

The Navier-Stokes (NS) equations arise from the principles of conservation of mass, momentum and energy. They help describe the motion of fluid particles for example changes in weather patterns, aerodynamics of a wing, flow through a pipe and especially the compressible version of NS equations can be used to model high-speed flows where shock wave formation occurs. . The overall NS equations can be written in as:

- Continuity Equation (mass conservation):

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0 \tag{1}$$

- Momentum Equation (momentum conservation):

$$\frac{\partial (\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u} \otimes \mathbf{u}) = -\nabla p + \nabla \cdot \mathbf{T} + \rho \mathbf{f} \tag{2}$$

  Here, $\mathbf{T}$ represents the stress tensor, $\mathbf{u}$ is the velocity vector, and $\mathbf{f}$ is the body force per unit mass.
- Energy Equation (energy conservation):

$$\frac{\partial (\rho E)}{\partial t} + \nabla \cdot (\mathbf{u}(\rho E + p)) = \nabla \cdot (\mathbf{u} \cdot \mathbf{T}) + \nabla \cdot (\kappa \nabla T) + \rho \mathbf{u} \cdot \mathbf{f} \tag{3}$$

  Where, E is the total energy per unit mass and κ is the thermal conductivity. where ρ is the fluid density, $\mathbf{u}$ is the fluid velocity, t is time, p is pressure, μ is viscosity, I is the identity matrix,$\mathbf{f}$ is the body forces (generally taken as gravity), e is the total energy in the system, κ is the thermal conductivity and ΔT is the temperature gradient

To solve the incompressible version of the NS equation the energy equation is usually not considered. Also, since the flow is incompressible there is not change in density with time. i.e. in the continuity equation $\frac{\partial \rho}{\partial t} = 0$. The weak form formulation and the discretization is further discussed below in Methodology.

1.2 Problem Setup: Flow over a cylinder

The problem of fluid flow over a circular cylinder is a fundamental topic in fluid dynamics. When fluid like water or air encounters a cylindrical object, it is forced to separate and navigate around the object, resulting in formation of vortices and varying pressure distributions. The study of the flow over a cylinder has significance in various fields namely Aerospace Engineering, where the cylinder can be replaced by an airfoil cross section to to calculate lift and drag along it. The problem also serves as a benchmark for validating computational fluid dynamics models and techniques. It provides a basis for developing and testing numerical methods due to its well-documented flow characteristics and experimental data like DFG-Benchmark[1].

1.2.1 Objective

The objective of this project is to use this problem in 2-D setup to verify the accuracy and reliability of the incompressible NS formulation using the FEniCSx programming software.

1.2.2 Geometry

The geometry considered for this project is a cross-section of a pipe with a circular cross-section of a cylinder at a certain distance from the inlet. In this 2-D case, the pipe is of length 2.2 m and width 0.41 m. The cylinder is as 0.2 m distance from the inlet with a radius of 0.05 m at 0.2 m from the bottom. This geometry is similar to the geometry described in Benchmark setup for DFG flow around cylinder[1].

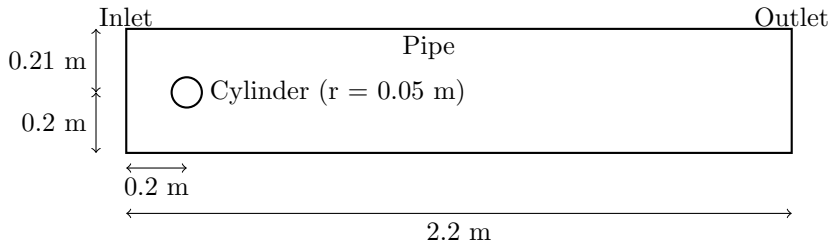

Fig. 1 Geometry setup for the flow over a cylinder problem

1.2.3 Boundary Conditions and inlet profile

No slip boundary condition are applied to the top and bottom walls. $u|_\Gamma = 0$ and at the inlet a parabolic inflow profile is provided with a maximum velocity given by a sine-function as described in DFG-Benchmark[1].

$$u(0, y) = \left(\frac{4Uy(0.41 - y)}{0.41^2}, 0 \right), \tag{4}$$

$$U = U(t) = 1.5sin(\pi t/8) \tag{5}$$

a do-nothing boundary condition is defined on the outlet.

2 Methodology

2.1 Governing Equations

For the incompressible flow in 2-D domain, the governing Navier-Stokes can be written as: In domain $\Omega \subset \mathbb{R}^2$ at time $]0, T[$, where $T > 0$

- **Continuity Equation: Mass Balance**

$$\nabla \cdot \mathbf{u} = 0 \; in \; \Omega \in]0, T[\tag{6}$$

- **Momentum Equation: Force Balance**

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = \nabla \cdot \bar{\sigma} + f \; in \; \Omega \in]0, T[\tag{7}$$

where,

$$\bar{\sigma} = -p\bar{I} + \bar{\mathbf{T}}$$
$$\bar{\mathbf{T}} = 2\mu\epsilon(\mathbf{u})$$
$$\epsilon = \frac{1}{2}[\nabla\mathbf{u} + (\nabla\mathbf{u})^T]$$

- **Essential Boundary Condition**

$$u = g \; on \; \Gamma_g \tag{8}$$

- **Natural Boundary Condition**

$$n \cdot \sigma = h \; on \; \Gamma_h \tag{9}$$

- **Initial Condition**

$$u(x, 0) = u_0 \; \forall \; \Omega \; at \; t = 0 \tag{10}$$

2.2 Weak form formulations

Using the Method of Weighted Residuals

$$\int_\Omega q\nabla \cdot \mathbf{u} \, d\Omega = 0 \quad \forall \; q \in \mathcal{V}_p \tag{11}$$

$$\int_\Omega \rho \left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} \right) \cdot \mathbf{w} \, d\Omega = \int_\Omega \nabla \cdot \sigma \cdot \mathbf{w} \, d\Omega + \int_\Omega f \cdot \mathbf{w} \, d\Omega \quad \forall \; \mathbf{w} \in \mathcal{V}_u \tag{12}$$

As the equations consists of two variables to solve in velocity and pressure the trial and weighting spaces are:

Trial Solution space:

$$\delta^h{}_u = \{\mathbf{u}^h | \mathbf{u}^h \in H^1, \mathbf{u}^h = g^h on \Gamma_g\}$$

$$\delta^h{}_p = \{p^h | p^h \in H^1\}$$

Weighting Space:

$$\mathcal{V}^h{}_u = \{\mathbf{w}^h | \mathbf{w}^h \in H^1, w^h = 0 \; on \; \Gamma_g\}$$

$$\mathcal{V}^h{}_p = \{q^h | q^h \in H^1\}$$

This can be written as[2] :

$$\int_\Omega \mathbf{w}^h \cdot \rho \left(\frac{\partial \mathbf{u}^h}{\partial t} + (\mathbf{u}^h \cdot \nabla)\mathbf{u}^h \right) d\Omega - \int_\Omega \nabla p \cdot \mathbf{w}^h \, d\Omega + \int_\Omega \mu(\nabla \mathbf{u}^h : \nabla \mathbf{w}^h) \, d\Omega$$
$$- \int_\Omega f \cdot \mathbf{w}^h \, d\Omega - \int_{\Gamma_h} h \cdot \mathbf{w}^h \, d\Gamma - \int_\Omega q\nabla \cdot \mathbf{u}^h \, d\Omega = 0 \tag{13}$$

2.3 Time Discretization

Generalized-alpha method is a unconditionally stable time integration schemes for nonlinear problem. This method uses spectral radius parameter ρ_∞ that controls numerical dissipation of high-frequency into the time integration scheme[3]. The spectral radius is mathematical concept that measures the amplification of numerical errors at infinite frequency. It ranges from 0 to 1, 0 being no numerical dissipation and 1 being the maximum. It introduces parameters like α_f, α_m and γ which are given as:

$$\begin{aligned}\alpha_f &= \frac{\rho_\infty}{1+\rho_\infty} \\ \alpha_m &= \frac{2\rho_\infty - 1}{1+\rho_\infty} \\ \gamma &= \frac{1}{2} + \alpha_m - \alpha_f\end{aligned}$$

Using these terms the velocity, pressure and acceleration (i.e. $\ddot{\mathbf{u}}$) parameters are update

$$\begin{aligned}\mathbf{u}^{n+1-\alpha_f} &= (1-\alpha_f)\mathbf{u}^n + \alpha_f \mathbf{u}^{n+1} \\ p^{n+1-\alpha_f} &= (1-\alpha_f)p^n + \alpha_f p^{n+1} \\ \ddot{\mathbf{u}}^{n+1-\alpha_m} &= (1-\alpha_m)\ddot{\mathbf{u}}^n + \alpha_m \ddot{\mathbf{u}}^{n+1}\end{aligned}$$

So, the time discretized weak formulation can be written as:

$$\begin{aligned}&\int_\Omega \rho \ddot{\mathbf{u}}^{n+1-\alpha_m} \mathbf{w}\, d\Omega + \int_\Omega \rho \frac{\mathbf{u}^{n+1-\alpha_f} - \mathbf{u}^n}{\Delta t} \mathbf{w}\, d\Omega + (1-\alpha_f)\int_\Omega \rho(\mathbf{u}^n \cdot \nabla)\mathbf{u}^n \mathbf{w}\, d\Omega \\ &+ \alpha_f \int_\Omega \rho(\mathbf{u}^{n+1} \cdot \nabla)\mathbf{u}^{n+1} \mathbf{w}\, d\Omega + (1-\alpha_m)\nu \int_\Omega \nabla \mathbf{u}^n : \nabla \mathbf{w}\, d\Omega + \alpha_m \nu \int_\Omega \nabla \mathbf{u}^{n+1} : \nabla \mathbf{w}\, d\Omega \\ &- (1-\alpha_f)\int_\Omega p^n \nabla \cdot \mathbf{w}\, d\Omega + \int_\Omega q \nabla \cdot \mathbf{u}^{n+1-\alpha_f}\, d\Omega - \alpha_f \int_\Omega p^{n+1} \nabla \cdot \mathbf{w}\, d\Omega \\ &= \int_\Omega \mathbf{f}^{n+1-\alpha_f} \mathbf{w}\, d\Omega \quad (14)\end{aligned}$$

2.4 Stabilizing terms

- Streamline Upwind Petrov-Galerkin (SUPG)
  SUPG is a type of discontinuous Galerkin element which provides more weightage to the upstream part of the element in order to address numerical instabilities due to the advection. It improves accuracy of solution near discontinuities and prevents spurious oscillations[4].

  $$\tau_{SUPG} = \frac{h}{2|\mathbf{u}|} \coth(Re)$$

  where, h is element size.

- Pressure Stabilized Petrov-Galerkin (PSPG)
  The PSPG term helps to address the instabilities associated with pressure-velocity coupling, allowing the use of equal-order interpolation for both pressure and velocity. Typically, the PSPG value is chosen to be the same as the SUPG term[5].

$$\tau_{PSPG} = \frac{h^2}{12\nu}$$

- Linear Stability Interior Crosswind (LSIC)
  The LSIC term is typically employed with low-order finite elements to mitigate numerical instabilities. It is particularly effective in reducing spurious oscillations and improving the overall convergence of the simulation[6].

$$\tau_{LSIC} = \frac{h}{2}$$

The following system of equations is added to the time discretized formulation equation 14.

$$\sum_{n_{el}} \int_{\Omega_{n_{el}}} \tau_{SUPG}[\vec{u}^{n+1-\alpha_f} \cdot \nabla\vec{v}][\vec{u}^{n+1-\alpha_f} \cdot \nabla\vec{u}^{n+1-\alpha_f}]\, d\Omega + \sum_{n_{el}} \int_{\Omega_{n_{el}}} \tau_{PSPG}[\vec{u}^{n+1-\alpha_f} \cdot \nabla\vec{v}]\nabla p^{n+1-\alpha_f}\, d\Omega + \sum_{n_{el}} \int_{\Omega_{n_{el}}} \tau_{LSIC}\nabla \cdot \mathbf{w}\nabla \cdot \mathbf{u}^{n+1-\alpha_f}\, d\Omega = 0 \quad (15)$$

where, n_{el} is the total number of elements.

2.5 Implementation in FEniCSx

After obtaining the required time discretized incompressible Velocity-Pressure couples NS equations along with the Stabilizing terms was encoded into FEniCSx software. The code used for the simulation is attached to the Appendix. This code is based on previously developed code by Dr. Chennakesava Kadapa[7].

To save time, the geometry and mesh generated for tutorial of the 2-D flow around the cylinder developed by Jørgen S. Dokken was utilized[8]. Certain changes were made to the "Facet tags" to match the mesh formation with the remaining part of the code. The generated mesh picture is shown in Fig.2.
Quadratic element was used. To show the working of the PSPG term same order function spaces and elements are used for both velocity and pressure. To work with two different order element mixed function space was also defined.

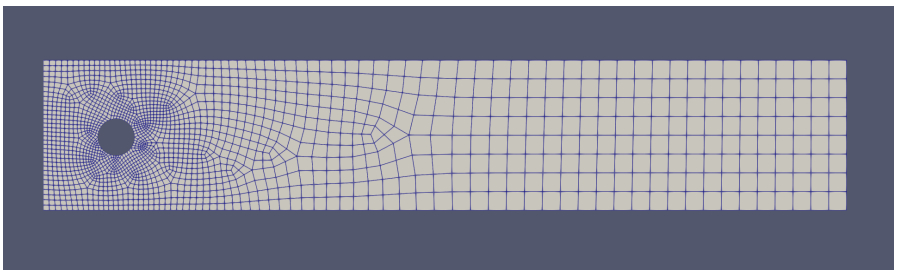

Fig. 2 Mesh generated using gmsh code from the tutorial

```
# Defining the volume integration measure "dx"
# also specifing the number of volume quadrature points.
dx = ufl.Measure('dx', domain=msh, \
metadata='quadrature_degree': 4})
ds = ufl.Measure('ds', domain=msh, \
subdomain_data=facet_tags, metadata={'quadrature_degree':4})

# FE Elements
# Quadratic element used
# Defining the order of function spaces and elements
deg_u = 1
deg_p = deg_u

# velocity
Ucg2 = element("Lagrange", msh.basix_cell(), deg_u, \
shape=(msh.geometry.dim,))
# pressure
Pcg1 = element("Lagrange", msh.basix_cell(), deg_p)

# Mixed element
Mcg = mixed_element([Ucg2, Pcg1])
ME = functionspace(msh, Mcg)

V2 = functionspace(msh, Ucg2) # Velocity function space
Q1 = functionspace(msh, Pcg1) # Pressure function space
```

After applying the previously dicussed boundary conditions, inlet velocity and time discretized formulation as required, problem was passed as a Nonlinear problem into Newton Solver and krylov solver was called upon to solve the problem. Tolerance of 1e-6 was given. Similar to the tutorial[8] calculation of coefficient of lift and drag were calculated for the cylinder and tallied with the data obtained from FEATFLOW simulation.

```
#setting up the cylinder as obstacle
n = -FacetNormal(msh)  # Normal pointing out of obstacle
obstacle_marker = 5
dObs = ufl.Measure("ds", domain=msh, \
subdomain_data=facet_tags, subdomain_id=obstacle_marker)
u_t = inner(as_vector((n[1], -n[0])), u)
# lift and drag calculation on the cylinder
drag = form(2 / 0.1 * (mu / rho * inner(grad(u_t), n)\
* n[0] - p * n[0]) * dObs)
lift = form(-2 / 0.1 * (mu / rho * inner(grad(u_t), n)\
* n[1] + p * n[1]) * dObs)
```

3 Results

The over all simulation was done for Reynold's number of 100 for total time of 8 seconds with a time step of 1/160 seconds. The result obtained from the simulation as shown in Fig.3 to 12 shows the development of flow and velocity and pressure fluctuation as desired.

Fig. 3 velocity at time step 2

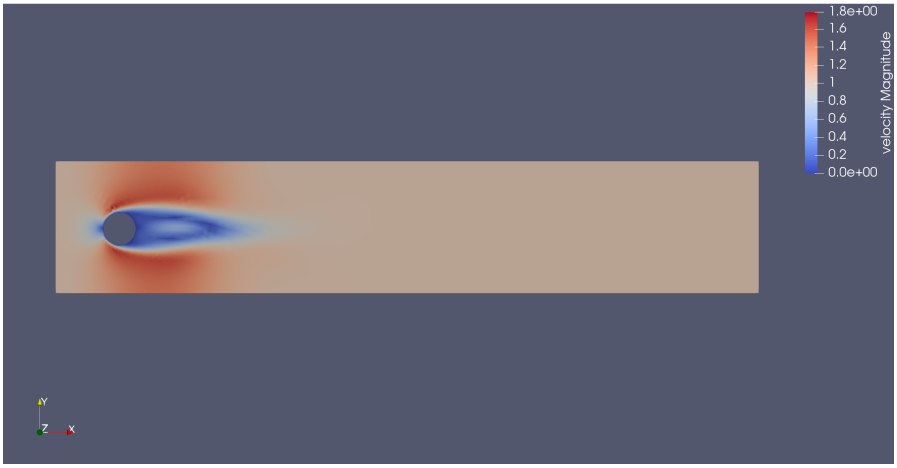

Fig. 4 velocity at time step 100

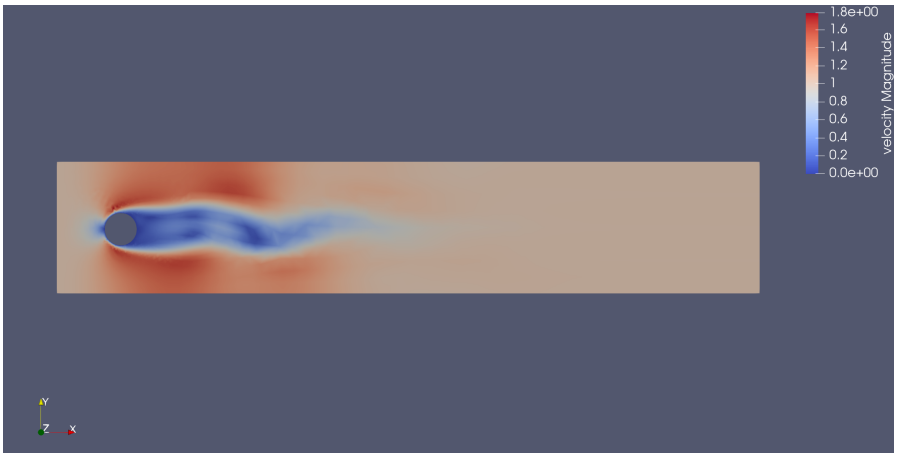

Fig. 5 velocity at time step 200

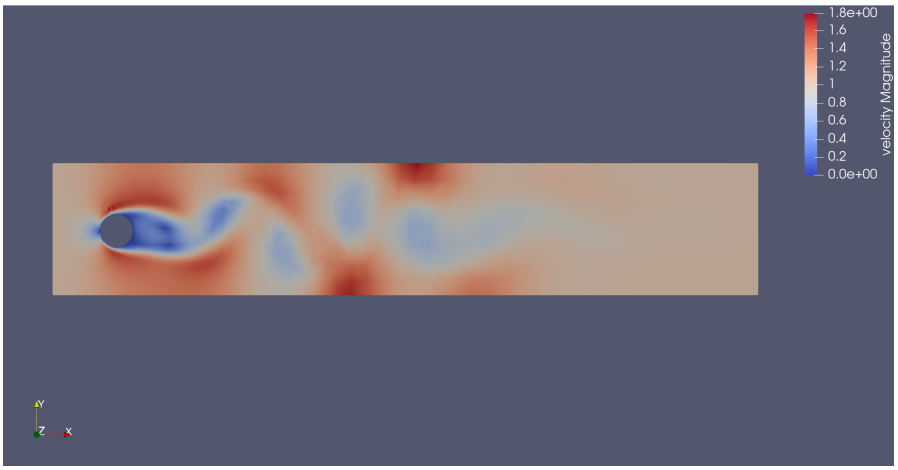

Fig. 6 velocity at time step 300

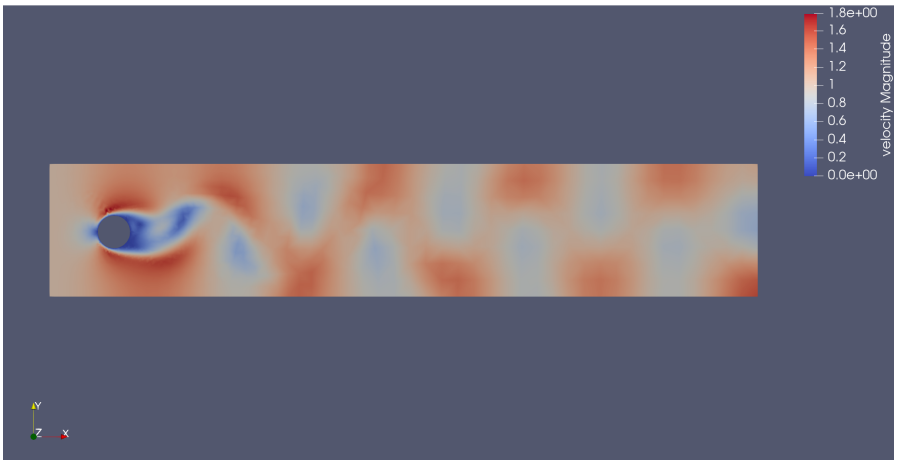

Fig. 7 velocity at time step 500 which continues to develop like this to the final time step

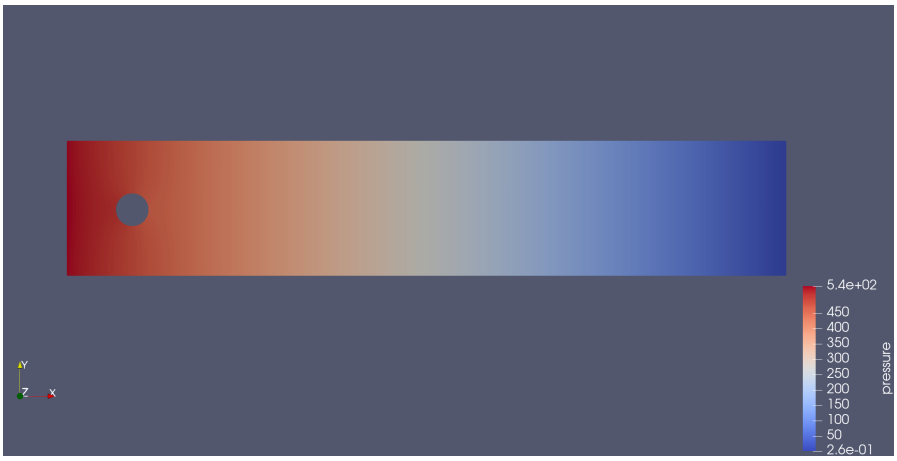

Fig. 8 pressure at time step 2

Fig. 9 velocity at time step 50

Fig. 10 velocity at time step 100

Fig. 11 velocity at time step 250

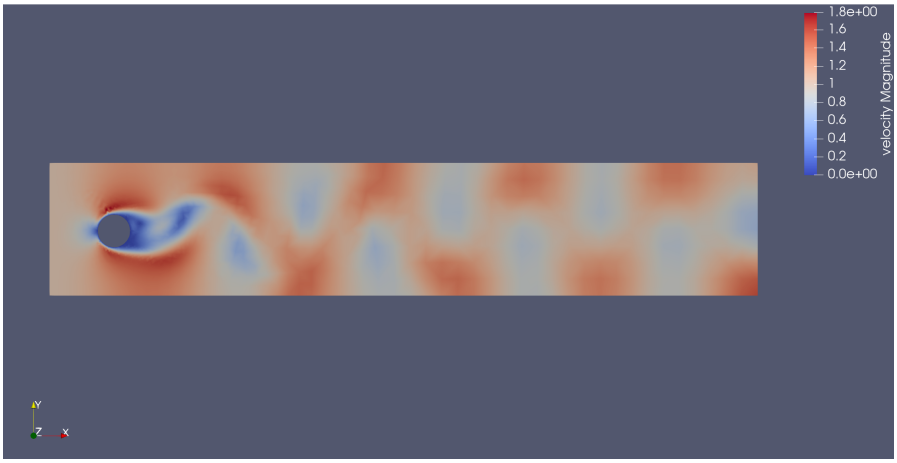

Fig. 12 velocity at time step 500 which continues to develop like this to the final time step

Similarly, the coefficient of lift and drag obtained from the simulations were also plotted against the DFG-Benchmark data for 2-D flow across the cylinder at Re=100[1], but the result obtained does not seem to match with the Benchmark data as seen in Fig. 13 and 14.

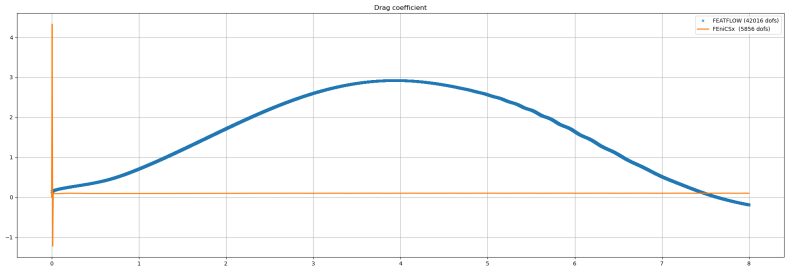

Fig. 13 Drag Coefficient comparison

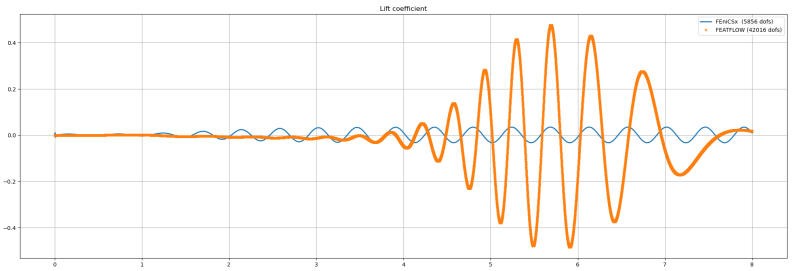

Fig. 14 Lift Coefficient comparison

4 Conclusion

The current version of the code seems to give highly outlying values at the first couple of time steps. So from the simulation and comparison with the Benchmark data it can be concluded that, even though the code is running and working for same order element due to the stabilizing and pressure correction terms the results obtained are not accurate.

The future work should be aimed at fixing the code to get accurate results so that it can be further optimized and energy and change in density terms could be added to develop a working code to solve the Compressible Navier Stokes.

References

[1] Turek, S., Schaefer, M.: Dfg benchmark 2d-3 flow around a cylinder (1996). Accessed: 2023-12-10

[2] Peterson, J.W., Lindsay, A.D., Kong, F.: Overview of the incompressible navier–stokes simulation capabilities in the moose framework. Advances in Engineering Software **119**, 68–92 (2018) https://doi.org/10.1016/j.advengsoft.2018.02.004

[3] Lovrić, A., Dettmer, W.G., Kadapa, C., Perić, D.: A new family of projection schemes for the incompressible navier–stokes equations with control of high-frequency damping. Computer Methods in Applied Mechanics and Engineering **339**, 160–183 (2018) https://doi.org/10.1016/j.cma.2018.05.006

[4] Brooks, A.N.: A petrov-galerkin finite element formulation for convection dominated flows. PhD thesis, California Institute of Technology (1981). https://thesis.library.caltech.edu/430/4/brooks_an_1981.pdf

[5] Tezduyar, T., Sathe, S.: Stabilization parameters in supg and pspg formulations. Journal of Computational and Applied Mathematics (2023). Accessed: 2024-12-10

[6] Liang, Y., Zhang, S.: Least-squares methods with nonconforming finite elements for general second-order elliptic equations. Journal of Scientific Computing (2023). Accessed: 2024-12-10

[7] Chennakesava: Cylinder2D-NS-coupled.py. Accessed: 2024-12-10 (2024). https://github.com/chennachaos/CFDwithFEniCSx/blob/main/cylinder2D/Cylinder2D-NS-coupled.py

[8] Dokken, J.S.: Navier-Stokes Equations - Code 2. Accessed: 2024-12-10 (2024). https://jsdokken.com/dolfinx-tutorial/chapter2/ns_code2.html

Appendices

The code used for this simulation is available in my Public git repository. https://github.com/BeeLoki/FEniCSx-for-NS.git